GPMPG Implementation Plan

Goals

- Refactor merci-sales into a general-purpose marketing promotions engine.
- Keep deployment as a single Docker container serving frontend and backend.
- Use a simple SQLite-first approach with a persistent local database.
- Generate fixed PDF outputs and persist only the current application state needed to recreate them.
- Support YAML as the repository-facing campaign data format so users can store campaign/business data in version control outside the runtime database.

Working Assumptions (to confirm)

- Early implementation may start with one active campaign path per business, but the schema and API must extend cleanly to many campaigns per business.
- Only the current state needs to be stored in the application database.
- PDF files are stored on disk in a configurable output directory that defaults to the current directory.
- Local configuration uses a simple repo-root `.config` file with `KEY=VALUE` entries.
- Chat history is transient local process state and is not persisted in the database.
- Frontend and backend are built in one repo and shipped in one container.
- A standalone data-manager app will be added for direct SQLite business/campaign viewing and creation, starting with a single-business/single-campaign MVP import path from YAML.
- A configurable `DATA_DIR` value in `.config` points to the repository-managed YAML tree and defaults to `./data`.
- YAML files are the repository-facing campaign/business format; SQLite is the runtime working store used by the application.
- On GUI startup, if `DATA_DIR` is configured, the application reads/syncs it automatically; if the directory is missing, the application creates it.
- If `DATA_DIR` is not configured, that is an error in MVP.
- Prompting the user to choose a data directory and updating `.config` without restart is a future enhancement, not part of MVP.

Step-by-Step Plan

Step 1: Repository Bootstrap and Baseline Contracts

Objective:

- Create baseline folder structure for `frontend/`, `backend/`, and `backend/schemas/`.
- Define shared API contracts and naming conventions for IDs, timestamps, status enums, and output paths.

Deliverables:

- `backend/schemas/` with SQLite schema initialization scripts.
- `backend/app/` FastAPI skeleton and DB session wiring.
- `frontend/` Next.js app shell with API client module.

- Config loading for a local `.config` file with output-directory default behavior.

Testing highlights:

- Smoke test that backend starts and `/health` returns 200.
- Smoke test that frontend can render and call backend health endpoint.
- Config test that missing `.config` falls back to the current directory.
- Config parser test for simple `KEY=VALUE` settings.

Phase gate:

- Confirm folder layout and naming before model implementation.

Step 2: Core Data Model (Business + Campaign)

Objective:

- Implement foundational schema for core entities and relationships.

Schema highlights:

- `businesses`: legal identity, display name, timezone, active flag.
- `business_contacts`: phone/email/website support for multiple contact methods.
- `business_locations`: address and hours data for one or many locations.
- `brand_themes`: color tokens, typography preferences, logo references.
- `campaigns`: campaign name, optional secondary key, objective, status, schedule window, and enough structure to support many campaigns per business.
- A simple current-state editing model rather than revision history tables.

Identity and lookup rules:

- Allow multiple campaigns with the same name for a business.
- Treat campaign identity as a business-scoped composite of `campaign_name` plus optional `campaign_key` rather than using a surrogate naming convention.
- When a user requests a campaign name that already exists, prompt them to either open an existing matching campaign or create a new one.
- If creating a new duplicate-name campaign, require a secondary key such as `2026`.

Testing highlights:

- DB constraints: required fields, foreign keys, status enums.
- Model tests: one business to one campaign in the early flow, plus a schema-level test that multiple campaigns per business are supported.
- CRUD tests for the three required operations: new business/new campaign, existing business/new campaign, existing business/modify campaign.
- Duplicate-name campaign tests covering name collision prompts and `campaign_name` plus `campaign_key` uniqueness per business.

Phase gate:

- Review ERD and current-state model before adding rendering and output entities.

Step 3: Promotion Composition Model

Objective:

- Support arbitrary promotion structure (not single hardcoded flyer format).

Schema highlights:

- `campaign_offers`: multiple offers per campaign with validity windows.
- `campaign_assets`: images/logos/legal copy/media metadata.
- `template_definitions`: reusable format definitions (flyer, poster, etc.).
- `campaign_template_bindings`: campaign-to-template selection and overrides.

Testing highlights:

- Validation tests for overlapping offer windows.
- Asset metadata tests (mime, dimensions, source type).
- Template binding tests with override precedence checks.

Phase gate:

- Verify that at least two distinct promotion shapes can be represented.

Step 4a: Standalone SQLite Data Manager + YAML Import MVP

Objective:

- Add a standalone app focused on direct business/campaign data operations against SQLite.
- Support YAML-backed prepopulation from a repository-managed data directory in MVP.

Scope for MVP:

- Implement Step 4a as a web UI route inside the existing frontend.
- User can select an existing business from a list.
- User can select an existing campaign for that business.
- UI is intentionally single-context: one business and one campaign active at a time.
- MVP is view-only in the UI; create/update UI flows are deferred to a later step.
- YAML-backed sample data supports at least one business and one campaign in the data directory.
- Non-filesystem-safe business or campaign names are treated as startup validation errors in MVP and reported before the program exits.

Schema and service highlights:

- Add YAML load/sync service with strict schema validation and deterministic DB hydration behavior.
- Support hierarchical data layout under `DATA_DIR`:
  - one subdirectory per business using the business display name
  - one business YAML file in that directory using the same name as the directory
  - one subdirectory per campaign beneath the business directory using the campaign name
  - one campaign YAML file in that directory using the same name as the directory
- Treat missing YAML fields as null/empty when syncing into SQLite.
- Expect YAML files to be complete when used to populate or refresh DB records.

- Keep schema version optional in YAML.
- External campaign identity is defined by business name + campaign name + optional qualifier field, not by a user-managed campaign-id field.
- Display name is stored as a YAML property and defaults to the directory/file name in MVP.
- Internal numeric database keys remain acceptable as hidden implementation details only; they are not exposed in normal user-facing data interfaces, displays, or YAML.

UI/UX highlights:

- Standalone data manager route/app entry separated from main campaign workflow.
- Left panel or selector flow: choose business from available synced businesses.
- Secondary selector flow: choose campaign for selected business.
- Read-only object viewer for business and campaign fields in MVP so sample data can be visually inspected.
- If startup validation fails for unsafe business/campaign names or malformed YAML, surface a clear error report before the program exits.

Test-data fixture for MVP:

- Sample Step 4a YAML data lives under `tests/data/merci/`.
- Business fixture: `tests/data/merci/merci.yaml`
- Campaign fixture: `tests/data/merci/mothersday/mothersday.yaml`

How to run Step 4a MVP with sample data:

- Ensure `.config` contains `DATA_DIR=./tests/data`.
- Start the backend and frontend normally after Step 4a implementation is complete.
- On GUI startup, the application should create the data directory if missing, read `DATA_DIR`, and sync the sample YAML into SQLite automatically.
- Open the Step 4a standalone data-manager route in the frontend.
- Select business `merci` from the business list.
- Select campaign `mothersday` from the campaign list.
- Verify the read-only detail view shows the business profile, brand theme, campaign metadata, offer, asset, and template binding data from the sample YAML files.

Testing highlights:

- YAML loader tests for valid file, missing required fields, and type mismatches.
- Sync tests for clearing fields to null/empty when omitted from complete YAML updates.
- Integration test for business selection, campaign selection, and single-campaign view state.

Phase gate:

- Confirm YAML-to-DB mapping, data-directory conventions, and standalone read-only inspection UI for single-business/single-campaign MVP.
- Confirm external identity rules for campaigns: business + campaign name + optional qualifier.

Step 4b: Main-App Advanced Editing Integration + YAML-Backed Editing

Objective:

- Support a simple chat-style interface that edits a campaign's current state without introducing unnecessary AI infrastructure.
- Expose standalone data-manager capabilities in the main app as an advanced editing feature.
- Add YAML-backed create/update flows in the main app so repository-managed files become the editable source material.

Schema and service highlights:

- Deterministic command/update layer that maps user chat requests into campaign field changes.
- Server-side validation so edits remain simple, explicit, and reproducible.
- Transient in-process chat state that survives until the user exits the process, without database persistence.
- YAML write service that updates business and campaign files in the configured data directory.
- Maintain one business YAML per business directory and one campaign YAML per campaign directory.
- Add future rename/normalization workflow for non-filesystem-safe names, including user choice of safe path name versus display name.
- Add database-versus-data-directory reconciliation flow on startup when differences are detected.
- Present the user with a difference report and options to:
    - sync the database from local YAML data
    - overwrite local YAML data from the database
    - change the data directory and sync that directory
    - quit without changing either side

Testing highlights:

- Unit tests for command parsing or update routing logic.
- Validation tests for edit requests that touch offers, dates, branding, and campaign metadata.
- Tests ensuring only current state is persisted and reflected in the rendered output.
- Session tests proving chat history is discarded on process restart while campaign state remains.
- YAML write tests for deterministic file output and round-trip compatibility with YAML loading.
- Reconciliation tests for added, removed, and modified YAML/business/campaign records against database state.

Phase gate:

- Confirm the chat interaction remains a thin editing interface rather than an AI-generation subsystem.
- Confirm advanced editing access path in the main app and successful YAML-backed write behavior.

Step 5: Artifact Pipeline (Fixed PDF Output)

Objective:

- Generate and track fixed printable PDF outputs.

Schema and service highlights:

- `generated_artifacts`: file type, storage path, checksum, output timestamp, and status.

- Render pipeline that converts business + campaign + template state into a fixed PDF.
- Output-directory service driven by `.config`, defaulting to the current directory.

Testing highlights:

- Golden-file style checks for deterministic PDF sections.
- Checksum and artifact status transition tests.
- Regression tests ensuring each artifact links to the current campaign state and is written to the configured disk location.

Phase gate:

- Validate output quality baseline on at least one flyer and one poster template.

Step 6: API Surface and Frontend Integration

Objective:

- Build CRUD and workflow endpoints and wire frontend flows.

API highlights:

- Business endpoints: create, update, list, detail.
- Campaign endpoints: create, update, list, detail.
- Chat edit endpoint: submit edit request and receive updated campaign state.
- Campaign lookup endpoint behavior that returns duplicate-name matches and drives the open-or-create prompt.
- YAML sync endpoints: load business/campaign YAML from the configured data directory and later persist updates back to YAML files.
- Artifact endpoints: render PDF, list outputs, download metadata.

Frontend highlights:

- Business profile management UI.
- Campaign builder UI with template and offer management.
- Chat-style campaign editing panel tied to deterministic update actions.
- Duplicate campaign-name resolution prompt that lets the user open an existing campaign or create a new keyed campaign.
- Advanced editing entry point that opens the standalone-style business/campaign data manager.
- YAML sync controls for repository-based campaign data flow.
- Artifact preview/download panel for fixed PDF outputs.

Testing highlights:

- API contract tests for the required create/modify flows.
- Frontend integration tests for create/edit/render/download flow.
- Frontend integration tests for YAML load/sync and later YAML write actions.
- E2E happy path for business setup to artifact generation.

Phase gate:

- Approve MVP user flow before performance and hardening pass.

Step 7: Security, Validation, and Operational Hardening

Objective:

- Add guardrails and production-safe behaviors appropriate for MVP.

Implementation highlights:

- Input validation with explicit schema rules and bounds.
- File upload restrictions and content-type verification.
- Structured logging and request IDs for tracing.
- Startup readiness checks so tests do not race startup.

Testing highlights:

- Negative tests for invalid payloads and file uploads.
- Security-focused tests for path traversal and malformed media metadata.
- Readiness test to ensure integration tests wait for service health.

Phase gate:

- Confirm minimum security and observability checklist.

Step 8: Containerization and Local Runtime Workflow

Objective:

- Ensure reproducible local run, test, and build experience.

Implementation highlights:

- Multi-stage Dockerfile (frontend build output copied into backend static dir).
- Startup command initializes the SQLite schema if needed, then serves the app.
- Local scripts for backend tests, frontend tests, and e2e using dedicated port 3100.

Testing highlights:

- Docker build and run validation.
- Container smoke test for API + static frontend serving.
- E2E against containerized runtime for one complete flow.

Phase gate:

- Approve image size and local startup/runtime behavior.

Step 9: Data Migration Path from merci-sales and Cutover

Objective:

- Complete a clean-break refactor from merci-sales while preserving rendered PDF parity.

Implementation highlights:

- Identify the exact input data and rendering rules required to reproduce the current PDF.

- Rebuild those rules inside the generalized model without preserving old storage structures.
- Add parity fixtures that prove the refactored system can emit the same PDF output.
- Ensure parity scenarios can be represented through repository YAML data inputs used by GPMPG users.

Testing highlights:

- Golden-output comparison against representative merci-sales PDFs.
- Content-level assertions for layout-critical text, numbers, dates, and branding.
- Spot-check parity for the initial refactored promotion use case before extending templates.

Phase gate:

- Sign-off on migration quality and go-live readiness.

Testing Strategy (Detailed)

Test Pyramid

- Unit tests (largest share): domain validation, mapping logic, edit routing, render preparation, status transitions.
- Integration tests: FastAPI + SQLite in-memory for endpoint and DB behavior.
- E2E tests: Playwright flow on port 3100 from profile setup to chat-driven campaign edit to PDF generation.
- Data portability tests: YAML load/write round-trip checks against canonical fixtures.

Coverage Targets

- New feature target: aim for 80% coverage on the feature introduced in each planning step when useful and practical.
- Relax the target where hitting 80% would force low-value tests.
- Mandatory coverage areas: campaign CRUD flows, render pipeline, output-path handling, edit flow validation, and YAML load/write core logic.

Key Test Case Highlights

- Business and campaign lifecycle:
  - Create business with multiple locations and contacts.
  - Create first campaign for a new business.
  - Create additional campaign for an existing business once multi-campaign support is enabled.
  - Modify an existing campaign for an existing business.
  - Attempt to create a duplicate-name campaign and verify the user is prompted to open an existing campaign or supply a secondary key.
  - Reopen campaigns by name and verify keyed duplicates are disambiguated correctly.
- YAML data flow:
  - Load single-business/single-campaign YAML from the configured data directory to prepopulate DB.
  - Re-load same YAML and verify deterministic update behavior.

    - Later write selected business/campaign data back to YAML and verify round-trip compatibility.
    - Reject startup when business or campaign directory/file names are not filesystem-safe in MVP.
    - Later detect differences between YAML data and database state and present correct reconciliation options.
- Promotion composition:
    - Multiple offers with date constraints and conflict validation.
    - Template override precedence between business defaults and campaign overrides.
- Chat-driven editing:
    - User edit request updates the correct campaign fields.
    - Invalid edit request is rejected with actionable validation output.
    - Current-state persistence reflects the latest accepted edit only.
- Artifacts:
    - PDF generation success path with checksum stored.
    - Failed render transitions artifact status correctly.
    - Artifact is written to the configured output directory or current-directory default.
- Security and validation:
    - Reject oversized uploads and unsupported mime types.
    - Reject malformed URLs and invalid schedule windows.
    - Prevent unsafe file path writes.
- Runtime and container:
    - Service readiness endpoint gates integration/e2e start.
    - Container serves backend API and frontend static assets together.

Risks and Mitigations

- Risk: Overfitting schema to current flyer use case.
    - Mitigation: Keep offer/template/asset entities normalized and loosely coupled.
- Risk: Chat interface grows into an unnecessary AI subsystem.
    - Mitigation: Keep edit handling deterministic and limited to explicit state updates.
- Risk: YAML drift between repository files and runtime DB shape.
    - Mitigation: Define versioned YAML schema and enforce strict validation with actionable errors.
- Risk: User-facing business/campaign names conflict with filesystem-safe naming requirements.
    - Mitigation: Fail fast in MVP; add rename/normalization workflow in a later step while preserving display_name in YAML.
- Risk: SQLite-to-RDS migration friction.
    - Mitigation: Keep schema straightforward and avoid SQLite-only shortcuts in the core model.

Suggested Execution Mode

- Phase-by-phase implementation with explicit approval after each phase gate.
- Do not proceed to the next phase until current phase validation passes and you approve.